

CS301 — Advanced Graph Algorithms

CS301 — Algorithms & Data Structures

Slide 1: Slide 1: Advanced Graph Algorithms

Topics: Minimum Spanning Trees, Network Flow, NP-Completeness basics

Slide 2: Slide 2: Minimum Spanning Trees (MST)

MST: subset of edges connecting all vertices with minimum total weight.

Kruskal's Algorithm:

1. Sort edges by weight
2. Add edge if it doesn't create cycle (use Union-Find)

Time: $O(E \log E) = O(E \log V)$

Prim's Algorithm:

1. Start from any vertex
2. Greedily add cheapest edge connecting tree to non-tree vertex (use Priority Queue)

Time: $O(E \log V)$ with binary heap, $O(E + V \log V)$ with Fibonacci heap

Slide 3: Slide 3: Union-Find (Disjoint Set Union)

Efficient data structure for Kruskal's and cycle detection.

Operations:

- $\text{find}(x)$: return root of x 's component
- $\text{union}(x, y)$: merge components

Optimizations:

1. Union by rank: attach smaller tree under larger ($O(\log n)$)
2. Path compression: make all nodes point directly to root

With both: $O(\alpha(n))$ amortized per operation — essentially $O(1)$.

$\alpha(n)$ is the inverse Ackermann function, grows extremely slowly.

Slide 4: Slide 4: Network Flow

Max-Flow: find maximum flow from source s to sink t in a weighted directed graph.

Ford-Fulkerson Method:

1. Find augmenting path in residual graph (BFS for Edmonds-Karp)
2. Increase flow along path by bottleneck capacity
3. Repeat until no augmenting path

Edmonds-Karp (BFS-based Ford-Fulkerson): $O(VE^2)$

Max-Flow Min-Cut Theorem: max flow = min cut capacity.

Slide 5: Slide 5: NP and NP-Completeness

P: problems solvable in polynomial time

NP: problems where solution verifiable in polynomial time

NP-Complete: hardest problems in NP; every NP problem reduces to them

Known NP-Complete problems:

- Boolean Satisfiability (SAT) — Cook-Levin theorem
- 3-SAT
- Vertex Cover
- Hamiltonian Cycle
- Travelling Salesman Problem (decision version)
- 0-1 Knapsack (decision version)

P vs NP: unsolved! If $P=NP$, all these become polynomial time solvable.

Slide 6: Slide 6: Approximation Algorithms

For NP-hard optimization problems, we can't always find optimal solution efficiently.

Approximation: find solution within factor α of optimal.

Example: Vertex Cover 2-approximation:

1. Find any maximal matching M
2. Include both endpoints of every edge in M

Result: $\leq 2 \times \text{OPT}$ (polynomial time)

TSP with triangle inequality: Christofides' algorithm gives 1.5-approximation.

General TSP: no constant approximation unless $P=NP$ (unless exponential is ok).

Slide 7: String Algorithms

Pattern Matching:

- Naive: $O(nm)$ — try every position
- KMP (Knuth-Morris-Pratt): $O(n+m)$ — precompute failure function
- Rabin-Karp: $O(n+m)$ average using rolling hash
- Boyer-Moore: $O(n/m)$ best case — bad character + good suffix heuristics

Suffix Arrays + LCP Arrays: advanced string processing in $O(n \log n)$ or $O(n)$.

Slide 8: Slide 8: Amortized Analysis

Amortized analysis: average cost per operation over a sequence of operations.

Methods:

1. Aggregate: total cost / n operations
2. Accounting: charge more for cheap ops to 'bank' credit for expensive ops
3. Potential method: define potential Φ ; amortized cost = actual cost + $\Delta\Phi$

Example: Dynamic array (doubling)

- Occasional resize costs $O(n)$ but amortized cost per push is $O(1)$
- With potential $\Phi = 2(\text{size} - \text{capacity}/2)$: amortized cost of push = $O(1)$